claude-windows / 662b86ab-d2e3-4c02-8a0d-ec7a9071aba8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\662b86ab-d2e3-4c02-8a0d-ec7a9071aba8\subagents\workflows\wf_34671ac1-e8e\agent-a5595fdab525ba09e.jsonl`
- SHA-256: `567fe15896cb6c449c990332426020feeafe3b1f79af12008f326c2028127575`
- Source modified: `2026-06-21T10:21:07+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_34671ac1-e8e`
- session_id: `662b86ab-d2e3-4c02-8a0d-ec7a9071aba8`

Transcript

1. user (2026-06-21T10:18:25.336Z)

Review the COMPUTE_DECOUPLED_SERVING **M5** change just committed on branch feat/serving-m5-truly-local-checks (HEAD). Inspect it with: git diff origin/master...HEAD , and read the full files:
- backend/config/startup.py (NEW ? the per-profile fail/warn-fast checks)
- backend/config/presets.py (added probe_cuda_available)
- backend/config/__init__.py (exports)
- backend/main.py (server main(): enforce after global_config load; sys.exit(1) on fatal)
- backend/worker/__main__.py (_run_startup_checks helper; called in cmd_infer + cmd_train after _apply_overrides; returns rc 2 on fatal)
- tests/test_startup_checks.py (NEW, L1) + tests/test_profile_parity.py (NEW, L4)
The contract (must hold): DEFAULT-OFF ? when deployment.profile=="" NOTHING changes vs pre-M5 (byte-identical). Only an explicitly-selected profile triggers checks. cloud-serving + non-cloud storage = the ONLY fatal error (everything else is a warning) deliberately, to avoid false-positives (the CUDA probe is best-effort; ADC creds can't be confirmed offline). One isolated PR, default-OFF discipline. Report ONLY real, grounded issues (file:line). If a lens finds nothing real, return an empty findings array ? do not invent nits.

LENS = PRODUCTION FALSE-POSITIVES / FALSE-NEGATIVES of the checks. (1) STORAGE_INCOHERENT is the only fatal ? could a LEGITIMATE cloud-serving config ever trip it wrongly (e.g. a valid setup using a non-gcs/s3 backend)? Is gcs/s3 the right allowlist? (2) The CUDA warnings: probe_cuda_available() lazily imports torch in the WORKER process ? if the real GPU work happens in a native-WASB SUBPROCESS with its own torch env, the main process probe returns False and emits CUDA_NOT_VISIBLE on a box that DOES have a GPU. Is the WARNING-not-error choice + message wording honest about that? Could it confuse? (3) CREDS_UNCONFIRMED env hints (_GCP_ENV_HINTS) ? are they sufficient/over-broad? (4) Does the worker probe add a meaningful torch-import startup cost to EVERY infer/train, even profile=""? (Check: is probe called before or after the profile gate?) Find real false-positive/false-negative or cost issues.

2. assistant (2026-06-21T10:18:28.302Z)

I'll review the M5 change. Let me start by examining the diff and reading the relevant files.

3. user (2026-06-21T10:18:30.129Z)

```
6f78acd feat(serving): M5 ? per-profile startup checks + L4 profile-parity test (default-OFF)
fefa412 docs(serving): resolve §8 design picks (D16/D17) + reconcile per-video P3 vs M3 (#285)
06ff4b7 test(api): lock gs://|gcs:// /api/process -> queryable API-DB video_id (B-P1 guard)
(#284)
---DIFF STAT---
 backend/config/__init__.py                      |  13 +-
 backend/config/presets.py                       |  12 ++
 backend/config/startup.py                       | 179 ++++++++++++++++++++
 backend/main.py                                 |  18 +-
 backend/worker/__main__.py                      |  23 ++-
```

```
 .../runlogs/RUNLOG_truly-local_TEMPLATE.md         |  49 ++++++
 tests/test_profile_parity.py                       | 139 +++++++++++++++
 tests/test_startup_checks.py                       | 181 ++++++++++++++++++++
 8 files changed, 611 insertions(+), 3 deletions(-)
```

4. user (2026-06-21T10:18:32.352Z)

fetched

5. assistant (2026-06-21T10:18:34.262Z)

Let me read all the files in the diff.

6. user (2026-06-21T10:18:35.439Z)

```
diff --git a/backend/config/__init__.py b/backend/config/__init__.py
index 493c535..9b276c0 100644
--- a/backend/config/__init__.py
+++ b/backend/config/__init__.py
@@ -16,7 +16,13 @@ from .models import (  # re-export common ones
     StorageConfig,
     ValidationConfig,
 )
-from .presets import PROFILE_PRESETS, suggest_profile
+from .presets import PROFILE_PRESETS, probe_cuda_available, suggest_profile
+from .startup import (
+    StartupCheck,
+    StartupCheckError,
+    enforce_startup_checks,
+    run_startup_checks,
+)

 __all__ = [
     "load_config",
@@ -30,4 +36,9 @@ __all__ = [
     "ValidationConfig",
     "PROFILE_PRESETS",
     "suggest_profile",
+    "probe_cuda_available",
+    "StartupCheck",
+    "StartupCheckError",
+    "enforce_startup_checks",
+    "run_startup_checks",
 ]
diff --git a/backend/config/presets.py b/backend/config/presets.py
index f793227..5d49333 100644
--- a/backend/config/presets.py
+++ b/backend/config/presets.py
@@ -94,6 +94,18 @@ def _cuda_available() -> bool:
         return False


+def probe_cuda_available() -> bool:
+    """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
+
+    Exposed for the per-profile startup checks (``backend/config/startup.py``) so a caller
+    that *wants* the GPU heads-up can pass the result in without the startup module importing
+    torch itself. Best-effort by design: if torch lives only in a detector subprocess env
+    (native WASB), this returns ``False`` in the main process ? which is why the startup checks
+    treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a hard
+    error that could false-fail."""
+    return _cuda_available()
+
+
```

```
 def suggest_profile(
     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
 ) -> str:
diff --git a/backend/config/startup.py b/backend/config/startup.py
new file mode 100644
index 0000000..105688f
--- /dev/null
+++ b/backend/config/startup.py
@@ -0,0 +1,179 @@
+"""Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
+
+A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a coherent
+runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
+(expensive) job runs ? when the active profile is inconsistent with the environment:
+
+  * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one config bug
we
+    can detect with certainty + zero false-positive risk: a cloud profile that can't reach a
+    bucket can never serve). This is the README §4.3 "loud error before any job".
+  * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
+    falling back to CPU) ? a **warning**, never an error: the probe is best-effort (torch may
live
+    only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the
detector's
+    own healthcheck is the real authority. A warning surfaces it early without false-failing.
+  * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on Cloud
Run) ?
+    a **warning**: Application Default Credentials can come from the metadata server / gcloud,
which
+    we can't probe offline, so the storage backend's own init is the authority; we just flag
it.
+
+**DEFAULT-OFF:** with no profile selected (``profile == ""``) this is a strict **no-op** ?
exactly
+the pre-M5 behaviour. Only an explicitly-selected profile triggers any check (mirrors the
loader,
+which applies a preset only for an explicit profile). So nothing changes for today's manual-
knob
+deployments.
+"""
+
+from __future__ import annotations
+
+import logging
+import os
+from dataclasses import dataclass
+from typing import Any, Mapping, Optional
+
+LEVEL_ERROR = "error"
+LEVEL_WARN = "warn"
+
+# Storage backends that count as "a cloud bucket" for cloud-serving coherence.
+_CLOUD_STORAGE = ("gcs", "s3")
+# Env signals that imply Google ADC *might* resolve without an explicit creds file.
+_GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST",
+                  "CLOUDSDK_CONFIG", "GOOGLE_CLOUD_PROJECT")
+
+
+@dataclass(frozen=True)
+class StartupCheck:
+    """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-up)."""
+
+    level: str
+    code: str       # stable, greppable code (e.g. "STORAGE_INCOHERENT")
```

```
+    message: str
+
+
+class StartupCheckError(RuntimeError):
+    """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL check."""
+
+    def __init__(self, failures: list[StartupCheck]):
+        self.failures = failures
+        super().__init__(
+            "deployment profile startup check failed: "
+            + "; ".join(f"[{c.code}] {c.message}" for c in failures)
+        )
+
+
+def _creds_discoverable(env: Mapping[str, str]) -> bool:
+    """True if *some* Google ADC source is hinted by the environment. Best-effort ? a False
here
+    only downgrades to a WARNING, never a hard failure (the storage init is the authority)."""
+    return any(env.get(k) for k in _GCP_ENV_HINTS)
+
+
+def run_startup_checks(
+    config: Any,
+    *,
+    cuda_available: Optional[bool] = None,
+    env: Optional[Mapping[str, str]] = None,
+) -> list[StartupCheck]:
+    """Return the per-profile findings for ``config`` (empty when no profile is selected).
+
+    ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ? GPU
checks
+    are skipped; the API server passes ``None``, the worker passes
``presets.probe_cuda_available()``).
+    Pure: no IO, no torch import ? the caller owns the probe so this stays importable
everywhere.
+    """
+    profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
+    if not profile:
+        return []  # DEFAULT-OFF: no profile ? no checks (identical to pre-M5).
+
+    e = os.environ if env is None else env
+    storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
+    indexing = getattr(config, "indexing", None)
+    detector_impl = getattr(indexing, "detector_impl", "auto")
+    device = getattr(getattr(indexing, "wasb", None), "device", "cuda")
+
+    checks: list[StartupCheck] = []
+
+    if profile == "truly-local":
+        # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud backend here
is
+        # an inconsistent (but not fatal) choice worth surfacing.
+        if storage_backend not in ("local", "gdrive"):
+            checks.append(StartupCheck(
+                LEVEL_WARN, "STORAGE_UNUSUAL",
+                f"profile 'truly-local' usually uses local/gdrive storage, but
storage.backend="
+                f"'{storage_backend}'. Reads will go to a remote backend.",
+            ))
+        # GPU heads-up (warning only ? the detector healthcheck is the authority; the probe is
+        # best-effort and may be False even when a subprocess detector env has CUDA).
+        if cuda_available is False:
+            if device == "cuda":
+                checks.append(StartupCheck(
+                    LEVEL_WARN, "CUDA_NOT_VISIBLE",
```

```
+                    "device='cuda' but no CUDA GPU is visible to this process. If the box truly
"
+                    "has no GPU the detector healthcheck will hard-fail ? set
indexing.wasb.device="
+                    "'auto' for the CPU fallback (M4).",
+                ))
+            elif device == "auto":
+                checks.append(StartupCheck(
+                    LEVEL_WARN, "CPU_FALLBACK",
+                    "device='auto' resolved to CPU (no CUDA GPU visible). Inference will run on
"
+                    "CPU ? correct, but much slower than a GPU.",
+                ))
+
+    elif profile == "cloud-serving":
+        # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't serve.
+        if storage_backend not in _CLOUD_STORAGE:
+            checks.append(StartupCheck(
+                LEVEL_ERROR, "STORAGE_INCOHERENT",
+                f"profile 'cloud-serving' requires a cloud storage backend "
+                f"({'/'.join(_CLOUD_STORAGE)}), but storage.backend='{storage_backend}'. "
+                "A cloud job cannot read/write a local-only backend.",
+            ))
+        elif storage_backend == "gcs" and not _creds_discoverable(e):
+            checks.append(StartupCheck(
+                LEVEL_WARN, "CREDS_UNCONFIRMED",
+                "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on
Application "
+                "Default Credentials. The GCS backend will fail loudly at init if ADC can't
resolve.",
+            ))
+        if device == "cpu":
+            checks.append(StartupCheck(
+                LEVEL_WARN, "CLOUD_CPU_DEVICE",
+                "profile 'cloud-serving' runs on a GPU (L4) but indexing.wasb.device='cpu' ? "
+                "detection will not use the GPU.",
+            ))
+        elif cuda_available is False and device in ("cuda", "auto"):
+            checks.append(StartupCheck(
+                LEVEL_WARN, "CLOUD_CUDA_NOT_VISIBLE",
+                "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to this
process.",
+            ))
+
+    elif profile == "cpu-mock":
+        # The mock profile is for CI/regression ? a real detector here defeats the purpose.
+        if detector_impl != "stub":
+            checks.append(StartupCheck(
+                LEVEL_WARN, "MOCK_NONSTUB_DETECTOR",
+                f"profile 'cpu-mock' normally uses detector_impl='stub', but detector_impl="
+                f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
+            ))
+
+    return checks
+
+
+def enforce_startup_checks(
+    config: Any,
+    *,
+    cuda_available: Optional[bool] = None,
+    env: Optional[Mapping[str, str]] = None,
+    raise_on_error: bool = True,
+    logger: Optional[logging.Logger] = None,
+) -> list[StartupCheck]:
+    """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
```

```
+    ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.
+
+    Returns the full findings list (so a caller can inspect/surface them). A no-op with no
logging
+    and no raise when no profile is selected (``run_startup_checks`` returns ``[]``)."""
+    log = logger or logging.getLogger(__name__)
+    checks = run_startup_checks(config, cuda_available=cuda_available, env=env)
+    for c in checks:
+        if c.level == LEVEL_ERROR:
+            log.error("[STARTUP] %s: %s", c.code, c.message)
+        else:
+            log.warning("[STARTUP] %s: %s", c.code, c.message)
+    errors = [c for c in checks if c.level == LEVEL_ERROR]
+    if errors and raise_on_error:
+        raise StartupCheckError(errors)
+    return checks
diff --git a/backend/main.py b/backend/main.py
index 75888d3..8e05ce9 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -29,7 +29,13 @@ from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.stitcher.compiler import VideoCompiler
 from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
 from backend.utils.paths import resolve_under_root, safe_output_filename, validate_input_path
-from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  # new
typed config
+from backend.config import (  # new typed config
+    load_config as load_app_config,
+    AppConfig,
+    StitchingConfig,
+    enforce_startup_checks,
+    StartupCheckError,
+)
 from backend.results import Failure  # standardized error/result contract
 from backend.reporting import emit_indexer_report
 from backend import storage  # M2: URI-aware input acquisition (local = identity passthrough)
@@ -574,6 +580,16 @@ def main():
     global global_config, db_instance
     global_config = load_app_config()  # returns typed AppConfig

+    # M5: per-profile startup checks ? fail fast on an incoherent ACTIVE profile (e.g. a
+    # cloud-serving profile pointed at local-only storage) BEFORE any work. A strict no-op when
+    # no deployment.profile is selected (the default), so today's behaviour is unchanged. No
GPU
+    # probe here (the server stays torch-light; the detector healthcheck is the GPU authority).
+    try:
+        enforce_startup_checks(global_config, logger=logger)
+    except StartupCheckError as e:
+        logger.error("[STARTUP] refusing to start: %s", e)
+        sys.exit(1)
+
     # The CLI --output-dir overrides the configured output directory. It now lives
     # on the typed model (OutputConfig.output_dir), so every consumer ? including
     # /api/compile ? reads the same value instead of a write-only runtime hack.
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index fafb35a..c644ede 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -25,7 +25,12 @@ import tempfile
 from typing import Any, List

 from backend import storage
-from backend.config import load_config
+from backend.config import (
+    StartupCheckError,
```

```
+    enforce_startup_checks,
+    load_config,
+    probe_cuda_available,
+)
 from backend.infrastructure.database import Database
 from backend.pipeline.segmenters.base import segmenter_registry
 from backend.pipeline.validators.base import registry as validator_registry
@@ -88,9 +93,23 @@ def _write_report_json(video_id: str, segments: List[dict], status: str,
path: s
         }, f, indent=2)


+def _run_startup_checks(config) -> bool:
+    """M5 per-profile startup checks for the worker (the compute process, so it probes CUDA
+    best-effort). Logs warnings; returns False on a FATAL coherence error so the caller aborts
+    cleanly (rc 2) rather than run an expensive job under an incoherent profile. A no-op
(returns
+    True) when no deployment.profile is selected."""
+    try:
+        enforce_startup_checks(config, cuda_available=probe_cuda_available(), logger=logger)
+        return True
+    except StartupCheckError:
+        return False  # already logged at ERROR by enforce_startup_checks
+
+
 def cmd_infer(args: argparse.Namespace) -> int:
     config = load_config(args.config) if args.config else load_config()
     _apply_overrides(config, args.set)
+    if not _run_startup_checks(config):
+        return 2
     storage_cfg = config.storage.model_dump()

     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
@@ -260,6 +279,8 @@ def cmd_train(args: argparse.Namespace) -> int:

     config = load_config(args.config) if args.config else load_config()
     _apply_overrides(config, args.set)
+    if not _run_startup_checks(config):
+        return 2
     storage_cfg = config.storage.model_dump()

     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
```

7. assistant (2026-06-21T10:18:37.788Z)

Now let me read the test files and the key context around the worker probe call to understand the gate ordering.

8. user (2026-06-21T10:18:38.714Z)

```
1       """Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
2
3       L1 unit coverage of ``backend/config/startup.py``: the fail/warn-fast per-profile
checks.
4       Pure ? no torch/GPU/network (the CUDA probe + env are injected). The cardinal invariant:
5       **no profile selected ? ZERO checks** (default-OFF, identical to pre-M5).
6       """
7       import logging
8
9       import pytest
10
11      from backend.config import AppConfig
12      from backend.config.startup import (
13          LEVEL_ERROR,
14          LEVEL_WARN,
```

```
        StartupCheckError,
        enforce_startup_checks,
        run_startup_checks,
    )


    def _cfg(profile="", backend="local", device="cuda", detector_impl="auto"):
        """Build an AppConfig with arbitrary profile/storage/device combos ? including ones
a
        preset would never produce (e.g. cloud-serving + local storage), to exercise the
coherence
        checks. model_validate fills every other field with its default."""
        return AppConfig.model_validate({
            "deployment": {"profile": profile},
            "storage": {"backend": backend},
            "indexing": {"detector_impl": detector_impl, "wasb": {"device": device}},
        })


    def _codes(checks):
        return {c.code for c in checks}


    # ---------------------------------------------------------------- default-OFF

    @pytest.mark.parametrize("backend,device,detector", [
        ("local", "cuda", "auto"), ("gcs", "cpu", "stub"), ("s3", "auto", "native"),
    ])
    def test_no_profile_is_strict_noop(backend, device, detector):
        """profile='' ? ZERO checks regardless of any other field (the pre-M5 behaviour)."""
        cfg = _cfg(profile="", backend=backend, device=device, detector_impl=detector)
        assert run_startup_checks(cfg, cuda_available=False, env={}) == []


    def test_enforce_no_profile_never_raises_and_returns_empty():
        cfg = _cfg(profile="", backend="local")
        assert enforce_startup_checks(cfg, cuda_available=False, env={}) == []


    # ---------------------------------------------------------------- truly-local

    def test_truly_local_local_storage_gpu_present_is_clean():
        cfg = _cfg(profile="truly-local", backend="local", device="cuda")
        assert run_startup_checks(cfg, cuda_available=True, env={}) == []


    def test_truly_local_cuda_strict_without_gpu_warns():
        cfg = _cfg(profile="truly-local", backend="local", device="cuda")
        checks = run_startup_checks(cfg, cuda_available=False, env={})
        assert _codes(checks) == {"CUDA_NOT_VISIBLE"}
        assert all(c.level == LEVEL_WARN for c in checks)  # never fatal ? healthcheck is
the authority


    def test_truly_local_auto_without_gpu_warns_cpu_fallback():
        cfg = _cfg(profile="truly-local", backend="local", device="auto")
        checks = run_startup_checks(cfg, cuda_available=False, env={})
        assert _codes(checks) == {"CPU_FALLBACK"}


    def test_truly_local_unprobed_cuda_skips_gpu_checks():
        """cuda_available=None (not probed, e.g. the API server) ? no GPU heads-up at
all."""
        cfg = _cfg(profile="truly-local", backend="local", device="cuda")
        assert run_startup_checks(cfg, cuda_available=None, env={}) == []
```

```
76
77
78      def test_truly_local_cloud_storage_warns_unusual():
79          cfg = _cfg(profile="truly-local", backend="gcs", device="auto")
80          checks = run_startup_checks(cfg, cuda_available=True, env={})
81          assert "STORAGE_UNUSUAL" in _codes(checks)
82
83
84      def test_truly_local_gdrive_storage_is_fine():
85          cfg = _cfg(profile="truly-local", backend="gdrive", device="cuda")
86          assert run_startup_checks(cfg, cuda_available=True, env={}) == []
87
88
89      # ------------------------------------------------------------- cloud-serving
90
91      def test_cloud_serving_local_storage_is_fatal():
92          cfg = _cfg(profile="cloud-serving", backend="local")
93          checks = run_startup_checks(cfg, cuda_available=True,
env={"GOOGLE_APPLICATION_CREDENTIALS": "/k"})
94          incoherent = [c for c in checks if c.code == "STORAGE_INCOHERENT"]
95          assert incoherent and incoherent[0].level == LEVEL_ERROR
96
97
98      def test_cloud_serving_gcs_without_creds_warns():
99          cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
100         checks = run_startup_checks(cfg, cuda_available=True, env={})
101         assert _codes(checks) == {"CREDS_UNCONFIRMED"}
102         assert all(c.level == LEVEL_WARN for c in checks)
103
104
105     def test_cloud_serving_gcs_with_creds_is_clean():
106         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
107         assert run_startup_checks(cfg, cuda_available=True,
108                                   env={"GOOGLE_APPLICATION_CREDENTIALS": "/k"}) == []
109
110
111     def test_cloud_serving_on_cloud_run_env_skips_creds_warn():
112         """K_SERVICE (Cloud Run) implies ADC via the metadata server ? no creds warning."""
113         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
114         assert run_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "rally"}) ==
[]
115
116
117     def test_cloud_serving_cpu_device_warns():
118         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cpu")
119         checks = run_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "rally"})
120         assert "CLOUD_CPU_DEVICE" in _codes(checks)
121
122
123     def test_cloud_serving_no_gpu_warns():
124         cfg = _cfg(profile="cloud-serving", backend="gcs", device="cuda")
125         checks = run_startup_checks(cfg, cuda_available=False, env={"K_SERVICE": "rally"})
126         assert "CLOUD_CUDA_NOT_VISIBLE" in _codes(checks)
127
128
129     # ------------------------------------------------------------------ cpu-mock
130
131     def test_cpu_mock_stub_is_clean():
132         cfg = _cfg(profile="cpu-mock", backend="local", detector_impl="stub")
133         assert run_startup_checks(cfg, cuda_available=False, env={}) == []
134
135
136     def test_cpu_mock_real_detector_warns():
137         cfg = _cfg(profile="cpu-mock", backend="local", detector_impl="native")
138         checks = run_startup_checks(cfg, cuda_available=False, env={})
```

```
139         assert "MOCK_NONSTUB_DETECTOR" in _codes(checks)
140
141
142     # ------------------------------------------------------------------- enforce
143
144     def test_enforce_raises_on_fatal_and_carries_failures(caplog):
145         cfg = _cfg(profile="cloud-serving", backend="local")
146         with caplog.at_level(logging.ERROR):
147             with pytest.raises(StartupCheckError) as ei:
148                 enforce_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "x"})
149         assert [c.code for c in ei.value.failures] == ["STORAGE_INCOHERENT"]
150         assert "STORAGE_INCOHERENT" in caplog.text  # logged loudly at ERROR before raising
151
152
153     def test_enforce_does_not_raise_on_warnings_only():
154         cfg = _cfg(profile="truly-local", backend="local", device="auto")
155         checks = enforce_startup_checks(cfg, cuda_available=False, env={})  # CPU_FALLBACK
warning
156         assert checks and all(c.level == LEVEL_WARN for c in checks)  # returned, not raised
157
158
159     def test_enforce_raise_on_error_false_returns_errors_without_raising():
160         cfg = _cfg(profile="cloud-serving", backend="local")
161         checks = enforce_startup_checks(cfg, cuda_available=True, env={"K_SERVICE": "x"},
162                                         raise_on_error=False)
163         assert any(c.code == "STORAGE_INCOHERENT" and c.level == LEVEL_ERROR for c in
checks)
164
165
166     # ---------------------------------------------------- worker wiring (fail fast before
any work)
167
168     def test_worker_cmd_infer_aborts_rc2_on_incoherent_profile(tmp_path):
169         """The worker's M5 hook aborts cleanly (rc 2) BEFORE any fetch/segmentation when the
active
170         profile is fatally incoherent. The cloud-serving preset sets storage=gcs;
config.json overrides
171         it back to local ? STORAGE_INCOHERENT ? fail fast."""
172         from backend.worker import __main__ as wmain
173
174         cfg = tmp_path / "config.json"
175         cfg.write_text('{"deployment": {"profile": "cloud-serving"}, "storage": {"backend":
"local"}}')
176         video = tmp_path / "in.mp4"
177         video.write_bytes(b"FAKE")
178         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
179             "infer", "--video-uri", str(video), "--out-uri", str(tmp_path / "out"),
180             "--config", str(cfg), "--scratch", str(tmp_path / "s"), "--segmenter",
"wasb_hybrid"]))
181         assert rc == 2
182
```

9. user (2026-06-21T10:18:38.805Z)

```
1       """Profile-parity ? the load-bearing decoupling proof (COMPUTE_DECOUPLED_SERVING M5,
L4).
2
3       The decoupling guarantee: the same `(trajectory + config)` produces the **same candidate
4       windows + the same segment ranges** regardless of the active deployment profile /
orchestration.
5       Detection is replaced by the deterministic CPU **stub** (no GPU/WSL/network), so ANY
divergence
6       is necessarily a profile/seam leak into the pure core ? exactly what D1 forbids.
7
8       Three arms:
```

```
9         * **L4a** ? the WINDOW stage (`trajectory_to_action_windows`) is a pure, profile-
agnostic
10          staticmethod: deterministic across calls + its signature carries no
profile/compute/storage
11          parameter (so it *structurally* cannot branch on a profile).
12        * **L4b** ? the real stub-backed segmenter produces byte-identical segment ranges
under
13          `profile=truly-local` vs `profile=cpu-mock` vs no profile (the only varied knob is
the
14          compute target, which the core never reads).
15        * **L4c** ? the worker (`cmd_infer`, the cloud-serving orchestration) writes an
`intervals.csv`
16          whose ranges match the in-process truly-local run for the same stub trajectory.
17      """
18      import inspect
19      import json
20
21      from backend.config import load_config
22      from backend.infrastructure.database import Database
23      from backend.pipeline.detectors.stub_runner import StubDetectorRunner
24      from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
25
26
27      # A fixed duration so process_video's early ffprobe check passes for the fake
(undecodable) video;
28      # applied identically to every arm so it can never be the source of a divergence.
29      _DURATION = 30.0
30
31
32      def _config(tmp_path, profile):
33          """A config.json applying `profile` (or none), forced to the deterministic CPU stub
+
34          fully-local (no AI handoff) + a tmp output dir. (The stub uses its default synth
params ?
35          `indexing.stub` is not a typed knob, so it deterministically self-configures.)"""
36          data = {
37              "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
38              "output": {"output_dir": str(tmp_path / "out")},
39          }
40          if profile:
41              data["deployment"] = {"profile": profile}
42          cfg_file = tmp_path / f"config_{profile or 'none'}.json"
43          cfg_file.write_text(json.dumps(data))
44          return load_config(cfg_file)
45
46
47      def _ranges(segments):
48          """Normalise a segment list to comparable (start, end) tuples (rounded to drop float
noise)."""
49          out = []
50          for s in segments:
51              start = s.get("start_time", s.get("start"))
52              end = s.get("end_time", s.get("end"))
53              out.append((round(float(start), 3), round(float(end), 3)))
54          return out
55
56
57      def _inprocess_ranges(tmp_path, profile, monkeypatch):
58          tmp_path.mkdir(parents=True, exist_ok=True)
59          # The fake video is undecodable; pin the duration probe so the early validity check
passes.
60          # Applied to the shared module fn ? identical for every profile arm (never a
divergence source).
61
monkeypatch.setattr("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
```

```
                            lambda p: _DURATION)
        db = Database(str(tmp_path / f"db_{profile or 'none'}.sqlite"))
        cfg = _config(tmp_path, profile)
        video = tmp_path / "in.mp4"
        video.write_bytes(b"FAKEVIDEO-PARITY")
        db.add_video("vidParity", str(video), "processed", [])  # FK target for
candidate/play rows
        seg = WasbHybridSegmenter(db, cfg)
        result = seg.process_video("vidParity", str(video))
        return _ranges(result)


    # ---------------------------------------------------------------- L4a (pure WINDOW
stage)

    def test_window_stage_has_no_profile_parameter():
        """Structural guard: the WINDOW stage (`trajectory_to_action_windows`) carries no
        profile/compute/storage parameter, so it *cannot* branch on a deployment profile ?
the
        precise leak D1 forbids. (Determinism across profiles is exercised end-to-end in
L4b.)"""
        sig = inspect.signature(StubDetectorRunner.trajectory_to_action_windows)
        banned = {"profile", "compute_target", "deployment", "storage", "backend"}
        assert not (banned & set(sig.parameters)), \
            f"WINDOW stage must not take a profile/compute/storage param; got
{list(sig.parameters)}"


    # ---------------------------------------------------------------- L4b (in-process
parity)

    def test_inprocess_segment_ranges_identical_across_profiles(tmp_path, monkeypatch):
        """Same stub trajectory + config under different profiles ? byte-identical segment
ranges.
        The profiles differ only in compute_target/label, which the pure core never
reads."""
        local = _inprocess_ranges(tmp_path / "a", "truly-local", monkeypatch)
        mock = _inprocess_ranges(tmp_path / "b", "cpu-mock", monkeypatch)
        none = _inprocess_ranges(tmp_path / "c", "", monkeypatch)
        assert local == mock == none
        assert len(local) >= 1   # the stub actually produced rallies (the test isn't
vacuous)


    # ---------------------------------------------------------------- L4c (worker
orchestration)

    def test_worker_intervals_match_inprocess(tmp_path, monkeypatch):
        """The worker (cmd_infer ? the cloud-serving orchestration) serialises an
intervals.csv whose
        ranges equal the in-process truly-local run for the same stub trajectory. Proves the
        fetch/store/serialise orchestration doesn't perturb the core."""
        from backend.worker import __main__ as wmain

        # In-process reference ranges (this also pins get_video_duration for the worker run
below).
        ref = _inprocess_ranges(tmp_path / "ref", "truly-local", monkeypatch)

        # Worker path: stub + skip-AI, validators mocked (fake bytes aren't a real video).
        class _OK:
            passed = True
            validator_name = "fake"
            message = "ok"

            def model_dump(self):
```

```
115                 return {"validator_name": "fake", "passed": True, "message": "ok"}
116
117         monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
118                             lambda path, cfg: ([_OK()], {}))
119         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
120
121         cfg_file = tmp_path / "wcfg.json"
122         cfg_file.write_text(json.dumps({
123             "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
124             "output": {"output_dir": str(tmp_path / "wout")},
125         }))
126         video = tmp_path / "w.mp4"
127         video.write_bytes(b"FAKEVIDEO-PARITY")
128         out = tmp_path / "bucket"
129
130         rc = wmain.cmd_infer(wmain.build_parser().parse_args([
131             "infer", "--video-uri", str(video), "--out-uri", str(out), "--config",
str(cfg_file),
132             "--scratch", str(tmp_path / "scratch"), "--segmenter", "wasb_hybrid"]))
133         assert rc == 0
134
135         # Find the intervals.csv (outputs/<video_id>/intervals.csv).
136         intervals = next(out.rglob("intervals.csv"))
137         rows = intervals.read_text().splitlines()[1:]  # drop header
138         worker = [(round(float(r.split(",")[0]), 3), round(float(r.split(",")[1]), 3)) for r
in rows]
139         assert worker == ref
140
```

10. assistant (2026-06-21T10:18:42.642Z)

Now let me verify the worker probe ordering concern. The key question for cost: is `probe_cuda_available()` called before or after the profile gate? Let me look at `_run_startup_checks` and how `enforce_startup_checks` ? `run_startup_checks` flows, plus the `_apply_overrides` ordering.

11. user (2026-06-21T10:18:44.542Z)

```
96      def _run_startup_checks(config) -> bool:
97          """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
98          best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
99          cleanly (rc 2) rather than run an expensive job under an incoherent profile. A no-op
(returns
100         True) when no deployment.profile is selected."""
101         try:
102             enforce_startup_checks(config, cuda_available=probe_cuda_available(),
logger=logger)
103             return True
104         except StartupCheckError:
105             return False  # already logged at ERROR by enforce_startup_checks
106
107
108     def cmd_infer(args: argparse.Namespace) -> int:
109         config = load_config(args.config) if args.config else load_config()
110         _apply_overrides(config, args.set)
111         if not _run_startup_checks(config):
112             return 2
113         storage_cfg = config.storage.model_dump()
114
115         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
116         os.makedirs(scratch, exist_ok=True)
117         config.output.output_dir = scratch
118
```

```
119         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
120         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
121         print(f"[worker] pulled {args.video_uri} -> {local_video}")
122
123         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
124         video_id = compute_video_id(local_video)
125
```

12. user (2026-06-21T10:18:44.983Z)

```
70
71      def preset_for(profile: str) -> dict[str, Any]:
72          """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73          module-level table), or ``{}`` for ``""`` / an unknown profile."""
74          return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77      def _truthy(val: Optional[str]) -> bool:
78          """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79          if val is None:
80              return False
81          return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84      def _cuda_available() -> bool:
85          """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86          ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87          try:
88              # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89              # and optional). Only reached when a caller opts into probe_cuda.
90              import torch
91
92              return bool(torch.cuda.is_available())
93          except Exception:
94              return False
95
96
97      def probe_cuda_available() -> bool:
98          """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100         Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101         that *wants* the GPU heads-up can pass the result in without the startup module
importing
102         torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103         (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104         treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105         error that could false-fail."""
106         return _cuda_available()
107
108
109     def suggest_profile(
```

13. assistant (2026-06-21T10:18:49.312Z)

This confirms a real cost issue. In the worker, line 102: `probe_cuda_available()` is the **argument** to `enforce_startup_checks`, so Python evaluates it **before** the call ? meaning torch is imported eagerly on **every** `cmd_infer`/`cmd_train`, even when `deployment.profile == ""` (default-OFF), where `run_startup_checks` returns `[]` immediately and the probe result is never used.

Let me check the native-WASB subprocess architecture to verify the false-positive concern about CUDA visibility, and check how the worker actually runs detection (subprocess vs in-process).

14. user (2026-06-21T10:18:51.440Z)

```
backend\pipeline\detectors\wasb_runner.py:1:"""WASB shuttle-trajectory runner (WSL2 subprocess
adapter).
backend\pipeline\detectors\wasb_runner.py:6:  3. runs it in the WSL `wasb` conda env -> a
Frame,Visibility,X,Y trajectory CSV,
backend\pipeline\detectors\wasb_runner.py:15:import subprocess
backend\pipeline\detectors\wasb_runner.py:21:from backend.pipeline.detectors.tracknet_runner
import to_wsl_mnt_path, TrackNetRunner
backend\pipeline\detectors\wasb_runner.py:22:from backend.pipeline.detectors.base import
DetectorRunner, WslCommandMixin, wsl_tilde_quote
backend\pipeline\detectors\wasb_runner.py:35:    conda_sh: str =
"~/miniconda3/etc/profile.d/conda.sh"
backend\pipeline\detectors\wasb_runner.py:36:    conda_env: str = "wasb"
backend\pipeline\detectors\wasb_runner.py:39:    wsl_stage_dir: str = "~/clips"
backend\pipeline\detectors\wasb_runner.py:40:    timeout_sec: int = 1800  # 30 min; a hung
GPU/conda call is killed (0 = no timeout)
backend\pipeline\detectors\wasb_runner.py:53:            distro=w.get("wsl_distro", "Ubuntu"),
backend\pipeline\detectors\wasb_runner.py:55:            conda_sh=w.get("conda_sh",
"~/miniconda3/etc/profile.d/conda.sh"),
backend\pipeline\detectors\wasb_runner.py:56:            conda_env=w.get("conda_env", "wasb"),
backend\pipeline\detectors\wasb_runner.py:60:            wsl_stage_dir=w.get("wsl_stage_dir",
"~/clips"),
backend\pipeline\detectors\wasb_runner.py:74:        cmd = (f"conda activate
{self.cfg.conda_env} && "
backend\pipeline\detectors\wasb_runner.py:78:            res = self._wsl_bash(cmd)
backend\pipeline\detectors\wasb_runner.py:80:            return False, "`wsl` not found ?
Windows + WSL2 required."
backend\pipeline\detectors\wasb_runner.py:81:        except subprocess.TimeoutExpired:
backend\pipeline\detectors\wasb_runner.py:89:        src_mnt = to_wsl_mnt_path(video_win_path)
backend\pipeline\detectors\wasb_runner.py:90:        reason =
self.wsl_source_unreadable_reason(src_mnt)
backend\pipeline\detectors\wasb_runner.py:94:        dest =
f"{self.cfg.wsl_stage_dir}/{dest_name}"
backend\pipeline\detectors\wasb_runner.py:95:        cmd = f"mkdir -p {self.cfg.wsl_stage_dir}
&& cp {shlex.quote(src_mnt)} {wsl_tilde_quote(dest)} && echo OK"
backend\pipeline\detectors\wasb_runner.py:97:            res = self._wsl_bash(cmd)
backend\pipeline\detectors\wasb_runner.py:98:        except subprocess.TimeoutExpired:
backend\pipeline\detectors\wasb_runner.py:108:        src_mnt = to_wsl_mnt_path(_INFER_WIN)
backend\pipeline\detectors\wasb_runner.py:110:        res = self._wsl_bash(cmd)
backend\pipeline\detectors\wasb_runner.py:122:        # translation: to_wsl_mnt_path passes
relative paths through unchanged, so
backend\pipeline\detectors\wasb_runner.py:133:        wsl_out_dir =
to_wsl_mnt_path(output_win_dir)
backend\pipeline\detectors\wasb_runner.py:142:        frames_dir =
f"{self.cfg.wsl_stage_dir}/{key}_frames"
backend\pipeline\detectors\wasb_runner.py:148:            free =
self._wsl_free_gb(self.cfg.wsl_stage_dir)
backend\pipeline\detectors\wasb_runner.py:153:                    free, self.cfg.wsl_stage_dir,
self.cfg.min_free_gb)
backend\pipeline\detectors\wasb_runner.py:160:            frame_args = f"--frames_out_dir
{wsl_tilde_quote(frames_dir)} "
backend\pipeline\detectors\wasb_runner.py:162:            f"conda activate {self.cfg.conda_env}
&& cd {self.cfg.repo_dir}/src && "
backend\pipeline\detectors\wasb_runner.py:164:            f"--video
{wsl_tilde_quote(staged_video)} "
```

```
backend\pipeline\detectors\wasb_runner.py:166:            f"--weights
{wsl_tilde_quote(self.cfg.weights_path)} "
backend\pipeline\detectors\wasb_runner.py:168:            f"--out {wsl_tilde_quote(wsl_out_dir +
'/' + key + '_wasb.csv')}"
backend\pipeline\detectors\wasb_runner.py:172:            res = self._wsl_bash(cmd)
backend\pipeline\detectors\wasb_runner.py:173:        except subprocess.TimeoutExpired:
backend\pipeline\detectors\wasb_runner.py:194:            self._wsl_rm_rf(stale)
backend\pipeline\detectors\wasb_infer.py:10:It must run INSIDE the WSL `wasb` conda env, from
the WASB-SBDT `src/` dir (it
```

15. user (2026-06-21T10:18:52.104Z)

```
Found 20 files limit: 20
tests\test_startup_checks.py
tests\test_profile_parity.py
backend\worker\__main__.py
backend\main.py
backend\config\startup.py
backend\config\presets.py
tests\test_native_wasb_runner.py
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\detectors\device.py
backend\config\models.py
tests\test_output_base.py
backend\pipeline\segmenters\trajectory_hybrid.py
tests\test_config_deployment.py
tests\test_worker.py
backend\utils\workspace.py
tests\test_workspace.py
tests\test_generate_highlights.py
tests\test_config_wasb.py
artifacts\so.py
backend\pipeline\segmenters\fusion_hybrid.py
```

16. assistant (2026-06-21T10:19:00.132Z)

```
This confirms the subprocess/WSL architecture: the WASB detector runs detection in a **WSL conda
`wasb` env subprocess** ? completely separate from the worker process's Python/torch. So the
main worker process very plausibly has no torch (or no CUDA-enabled torch) while the actual GPU
work happens in the subprocess. The design notes acknowledge this and chose WARNING. Let me
check the native_wasb_runner and device.py to understand the `device='auto'` M4 path and whether
the message wording is accurate.
```

17. user (2026-06-21T10:19:00.984Z)

```
1       """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3       The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4       "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5       It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6       Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py``
core
7       that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8       see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10        * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11          ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12          (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14          (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
```

```
15          strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16        * weights / repo / python / scratch come from the **environment** first
17          (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18          ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20      It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21      name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22      ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23      the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24      (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26      **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27      (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28      candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29      ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30      run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31      vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32      artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34      not the numbers.
35      """
36      import os
37      import shutil
38      import logging
39      import subprocess
40      from dataclasses import dataclass
41      from typing import Any, Dict, List, Optional, Tuple
42
43      from backend.pipeline.detectors.base import DetectorRunner
44      from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
45      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
46      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
47
48      logger = logging.getLogger(__name__)
49
50      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
51      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
52      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
53
54      # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
55      # so the "cuda True/False" string they parse can never drift apart.
56      _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
57
58
59      @dataclass
60      class NativeWasbConfig:
```

18. user (2026-06-21T10:19:01.521Z)

```
1       """DeviceContext ? resolve the effective torch device for a detector runner.
2
3       COMPUTE_DECOUPLED_SERVING M4 (realizes the ``DeviceContext`` designed in the archived
4       ``07-COMPUTE-ABSTRACTION``). A runner asks for a device by name; this turns the
```

```
*requested*
5       device + a probed CUDA availability into the *effective* device to actually run on, and
says
6       whether a CPU fallback happened.
7
8       The portability gap it closes: the native WASB runner hardcoded ``device='cuda'`` and
its
9       healthcheck hard-failed on a box with no GPU, so a CPU-only Linux box (cpu-serving / a
10      no-GPU truly-local install / CI) could not run at all.
11
12      **Safety contract (so "cuda default unchanged" holds):**
13        * ``"auto"``  ? ``cuda`` if a GPU is available, else ``cpu`` (the fallback). This is
opt-in.
14        * ``"cuda"``  ? honored verbatim; an *explicit* cuda request on a GPU-less box still
hard-fails
15          elsewhere (no silent slow-CPU downgrade of a config that asked for the GPU).
16        * ``"cpu"`` / ``"mps"`` ? honored verbatim.
17      """
18
19      from __future__ import annotations
20
21      from dataclasses import dataclass
22
23      AUTO = "auto"
24      CUDA = "cuda"
25      MPS = "mps"
26      CPU = "cpu"
27
28      # The device names a runner accepts. ``auto`` resolves; the rest are torch devices
passed verbatim.
29      VALID_DEVICES = (AUTO, CUDA, MPS, CPU)
30
31
32      @dataclass(frozen=True)
33      class DeviceContext:
34          """The requested device + a probed CUDA availability ? the effective device."""
35
36          requested: str          # what config/env asked for: auto | cuda | mps | cpu
37          cuda_available: bool     # probed in the runner's own torch env (not the main
process)
38
39          @property
40          def effective(self) -> str:
41              """The device the runner should actually use. ``auto`` resolves to cuda-if-
available-else-cpu;
42              any explicit device is returned verbatim."""
43              if self.requested == AUTO:
44                  return CUDA if self.cuda_available else CPU
45              return self.requested
46
47          @property
48          def fell_back(self) -> bool:
49              """True when ``auto`` resolved to CPU because no GPU was found (worth a loud
warning)."""
50              return self.requested == AUTO and not self.cuda_available
51
52          @property
53          def cuda_required_but_missing(self) -> bool:
54              """True when an EXPLICIT ``cuda`` request can't be satisfied ? the caller should
hard-fail
55              rather than silently downgrade (preserving the pre-M4 strict behaviour for
``device='cuda'``)."""
56              return self.requested == CUDA and not self.cuda_available
57
```

19. assistant (2026-06-21T10:19:08.366Z)

Now I have a critical confirmation. The `device.py` docstring (line 36) explicitly states: `cuda_available: bool # probed in the runner's own torch env (not the main process)`. This is the crux of the false-positive concern.

The key insight: the **native WASB runner probes CUDA in its own subprocess/env** (via `_CUDA_PROBE_CODE` run by `WASB_PYTHON`), which may be a different Python with CUDA-enabled torch. The M5 startup check's `probe_cuda_available()` probes torch in the **main worker process**. For the WSL runner, the main process has NO torch at all (it's all in the WSL conda env). So `CPU_FALLBACK` / `CUDA_NOT_VISIBLE` are emitted based on the wrong process's torch state.

Now let me scrutinize the `CPU_FALLBACK` message wording for accuracy. The message says `device='auto' resolved to CPU` and `Inference will run on CPU`. But the actual `auto` resolution happens in the runner's own env (`DeviceContext.cuda_available` = subprocess probe). So the main-process probe returning False does NOT mean inference resolved to CPU. Let me verify how the native runner builds its DeviceContext.

20. user (2026-06-21T10:19:09.943Z)

```
backend\pipeline\detectors\native_wasb_runner.py:44:from backend.pipeline.detectors.device
import AUTO, VALID_DEVICES, DeviceContext
backend\pipeline\detectors\native_wasb_runner.py:56:_CUDA_PROBE_CODE = "import torch;
print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
backend\pipeline\detectors\native_wasb_runner.py:107:        self._cuda_available:
Optional[bool] = None
backend\pipeline\detectors\native_wasb_runner.py:115:    # --- device resolution (M4
DeviceContext: device="auto" ? cuda-if-available-else-cpu) ------
backend\pipeline\detectors\native_wasb_runner.py:116:    def _probe_cuda(self) -> bool:
backend\pipeline\detectors\native_wasb_runner.py:120:            res =
self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
backend\pipeline\detectors\native_wasb_runner.py:127:        if self._cuda_available is None:
backend\pipeline\detectors\native_wasb_runner.py:128:            self._cuda_available =
self._probe_cuda()
backend\pipeline\detectors\native_wasb_runner.py:129:        return self._cuda_available
backend\pipeline\detectors\native_wasb_runner.py:131:    def _effective_device(self) -> str:
backend\pipeline\detectors\native_wasb_runner.py:135:            return DeviceContext(AUTO,
self._cuda_is_available()).effective
backend\pipeline\detectors\native_wasb_runner.py:173:            res =
self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
backend\pipeline\detectors\native_wasb_runner.py:182:        self._cuda_available = "cuda True"
in out
backend\pipeline\detectors\native_wasb_runner.py:183:        ctx =
DeviceContext(self.cfg.device, self._cuda_available)
backend\pipeline\detectors\native_wasb_runner.py:188:        if ctx.fell_back:
backend\pipeline\detectors\native_wasb_runner.py:225:        # M4: resolve device="auto" to the
effective device (cuda-if-available-else-cpu); an
backend\pipeline\detectors\native_wasb_runner.py:227:        device = self._effective_device()
```

21. assistant (2026-06-21T10:19:15.160Z)

Confirmed. The native runner's CUDA probe (line 120, 173) runs `self.cfg.python_bin` (the `WASB_PYTHON` env, a separate Python with the WASB-env torch), not the worker's torch. So the M5 main-process `probe_cuda_available()` and the runner's actual device resolution are **two independent probes against two different Python environments**. The worker-process probe returning `False` says nothing reliable about whether the detector will actually fall back to CPU.

Let me now verify the cost concern empirically ? confirm torch import is triggered eagerly even with profile="", and confirm there's no torch-light short-circuit. Let me also check whether torch is actually installed/heavy here.

22. user (2026-06-21T10:19:16.771Z)

```
275             DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
276             is the M3.5 "first real training" path; only the trajectory source flips.
277         """
278         import subprocess
279
280         config = load_config(args.config) if args.config else load_config()
281         _apply_overrides(config, args.set)
282         if not _run_startup_checks(config):
283             return 2
284         storage_cfg = config.storage.model_dump()
285
286         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
287         labels_dir = os.path.join(scratch, "labels")
288         traj_dir = os.path.join(scratch, "trajectories")
289         videos_dir = os.path.join(scratch, "videos")
290         os.makedirs(labels_dir, exist_ok=True)
291         os.makedirs(traj_dir, exist_ok=True)
292
293         corpus = args.corpus_uri.rstrip("/")
294         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
```

23. assistant (2026-06-21T10:19:19.682Z)

Now let me confirm the cost issue with a concrete test: does calling `_run_startup_checks` with `profile=""` actually import torch? Let me trace the evaluation order definitively and check whether torch is even installed in this env (to gauge severity).

24. user (2026-06-21T10:19:28.084Z)

```
torch in sys.modules before: False
probe_cuda_available() -> True took 3.146s
torch in sys.modules after: True
```

25. assistant (2026-06-21T10:19:34.233Z)

This is a concrete, measured confirmation. On this machine, `probe_cuda_available()` (the lazy torch import + `cuda.is_available()`) takes **3.1 seconds** and pulls torch into `sys.modules`. Because it is passed as a positional argument to `enforce_startup_checks(...)`, Python evaluates it **before** entering the function ? so it runs **on every `cmd_infer`/`cmd_train` invocation, including `profile==""` (default-OFF)**, where the result is immediately discarded (`run_startup_checks` returns `[]` before ever reading `cuda_available`).

This is a real default-OFF contract concern: the prompt asserts "when deployment.profile=='' NOTHING changes vs pre-M5 (byte-identical)". Behaviorally that holds for output, but operationally it adds a ~3s torch import + GPU init to every worker invocation that pre-M5 did not have. Let me confirm pre-M5 worker did not import torch on this path.

26. user (2026-06-21T10:19:35.996Z)

```
---exit 0---
```

27. assistant (2026-06-21T10:19:40.787Z)

Confirmed: pre-M5 `backend/worker/__main__.py` had **zero** torch references. So M5 introduces a ~3s torch import + CUDA init on every `infer`/`train`, even with no profile selected ? a real default-OFF regression in startup cost (and it can fail/hang on a box where torch import has side effects, though the `try/except` in `_cuda_available` swallows import errors).

Let me verify whether the worker process otherwise imports torch at all later in cmd_infer (if it inevitably loads torch anyway, the cost is moot). The detector runs in a subprocess, so the worker likely never imports torch otherwise.

28. user (2026-06-21T10:19:41.694Z)

```
No matches found
```

29. user (2026-06-21T10:19:42.672Z)

```
exit 0
```

30. assistant (2026-06-21T10:19:50.657Z)

Confirmed: the worker package never imports torch directly anywhere (detection is delegated to the WSL/native subprocess). So pre-M5, the worker process ran the entire `infer`/`train` job **without ever loading torch**. M5's eager probe now forces a torch import into the worker process unconditionally. On a CPU-only box where torch isn't installed, it's swallowed (fast); but on a GPU box with torch present, it's a real ~3s + GPU-context cost added to every job, including default-OFF.

Note the server path (`backend/main.py`) is fine ? it passes `cuda_available=None` (no probe), so the server stays torch-light. The cost issue is **worker-only**.

Now let me evaluate the `_GCP_ENV_HINTS` breadth (false-negative risk on CREDS_UNCONFIRMED). The hints include `GOOGLE_CLOUD_PROJECT` and `CLOUDSDK_CONFIG`, which are commonly set even when ADC won't resolve (e.g., `GOOGLE_CLOUD_PROJECT` set by an unrelated library, or `CLOUDSDK_CONFIG` pointing at a gcloud config dir with no active credentials). Let me confirm there's no s3 creds path and assess the allowlist question.

31. user (2026-06-21T10:19:52.266Z)

```
backend\config\presets.py:4:config knobs, so the truly-local / cloud-serving / cpu-mock modes
don't drift apart one
backend\config\presets.py:27:PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-
mock")
backend\config\presets.py:33:    "truly-local": "local_gpu",
backend\config\presets.py:34:    "cloud-serving": "cloud_run",
backend\config\presets.py:35:    "cpu-mock": "cpu_mock",
backend\config\presets.py:44:    "truly-local": {
backend\config\presets.py:45:        "deployment": {"profile": "truly-local", "compute_target":
"local_gpu"},
backend\config\presets.py:51:    "cloud-serving": {
backend\config\presets.py:52:        "deployment": {"profile": "cloud-serving",
"compute_target": "cloud_run"},
backend\config\presets.py:55:        "storage": {"backend": "gcs"},
backend\config\presets.py:57:    "cpu-mock": {
backend\config\presets.py:58:        "deployment": {"profile": "cpu-mock", "compute_target":
"cpu_mock"},
backend\config\presets.py:116:      - ``K_SERVICE`` set (Cloud Run)       ? ``"cloud-serving"``
backend\config\presets.py:117:      - ``CI`` truthy                       ? ``"cpu-mock"``
backend\config\presets.py:118:      - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
backend\config\presets.py:122:    imports torch; a CLI ``--detect`` or cloud-serving startup
that wants the GPU probe passes
backend\config\presets.py:127:        return "cloud-serving"
backend\config\presets.py:129:        return "cpu-mock"
backend\config\presets.py:131:        return "truly-local"
```

32. assistant (2026-06-21T10:19:57.958Z)

Let me check what storage backends are actually registered, to validate the `gcs/s3` allowlist for STORAGE_INCOHERENT (the one fatal). If there's any other legitimate cloud/remote backend registered (e.g. `http`, `azure`, a custom one), the fatal allowlist would false-positive.

33. user (2026-06-21T10:20:00.259Z)

```
backend\config\models.py:297:class StorageConfig(BaseModel):
backend\config\models.py:298:    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default
```

```
``local`` = today's
backend\config\models.py:304:    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
backend\config\models.py:315:    input_backend: Literal["local", "s3", "gcs", "gdrive"] =
"local"
backend\config\models.py:343:    (``indexing.detector_impl`` / ``skip_ai_handoff``,
``storage.backend``) in the loader.
```

34. user (2026-06-21T10:20:01.910Z)

```
backend\storage\base.py:22:    def fetch_to_local(self, ref: StorageRef, dst: "str |
os.PathLike[str] | None" = None,
backend\storage\gdrive.py:18:``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path.
This backend intentionally does NOT
backend\storage\gdrive.py:112:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\local.py:29:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\gcs.py:30:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) ->
str:
backend\storage\s3.py:38:    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
backend\storage\__init__.py:56:    return resolve_input_ref(raw, storage_config).backend ==
"local"
backend\storage\__init__.py:71:    if ref.backend == "local":
backend\storage\__init__.py:82:    if name == "s3":
backend\storage\__init__.py:85:    if name == "gcs":
backend\storage\__init__.py:88:    if name == "gdrive":
backend\storage\__init__.py:106:def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None,
*,
backend\storage\__init__.py:122:def list_uris(prefix_uri: str, *, config: Optional[Dict[str,
Any]] = None) -> Iterator[str]:
```

35. assistant (2026-06-21T10:20:07.966Z)

The storage backend is constrained by Pydantic `Literal["local", "s3", "gcs", "gdrive"]`. So the universe of backends is exactly 4. For STORAGE_INCOHERENT (cloud-serving fatal), the allowlist `(gcs, s3)` excludes `local` and `gdrive`.

Now, the false-positive question: is `gdrive` a legitimate cloud-serving backend? Let me check the gdrive backend ? it reads from `gcs://`/`s3://`/gdown. If a cloud-serving deployment legitimately used `gdrive` (which fetches from remote URIs including gcs/s3), STORAGE_INCOHERENT would fire fatally and refuse to start. Let me read the gdrive backend's actual capabilities to assess whether excluding it from the fatal allowlist is correct.

36. user (2026-06-21T10:20:09.051Z)

```
1       """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.
2
3       We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
Drive
4       for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
requirement ? it
5       needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
access. So
6       ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
7       your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
code the
8       ``local`` backend uses).
9
10          gdrive://Sharing/clip.mp4   ->   <mount>/My Drive/Sharing/clip.mp4
11
12      Why an alias instead of just a raw path? Portability: the mount root differs per machine
13      (Windows ``G:\\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
Drive``), so the same
```

```
14      ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
or set it
15      explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
secret).
16
17      A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
use
18      ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
intentionally does NOT
19      talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the
20      fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
21      """
22      from __future__ import annotations
23
24      import glob
25      import os
26      import string
27      from typing import Iterator, Optional
28
29      from backend.storage.base import StorageBackend
30      from backend.storage.local import LocalStorage
31      from backend.storage.ref import StorageRef, StorageStat
32
33      _INSTALL_URL = "https://www.google.com/drive/download/"
34
35      # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
36      # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
37      # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
38      _SYNC_REMEDY = (
39          "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
40          "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
41          "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
42          "downloads the bytes on first access."
43      )
44
45
46      def _not_mounted_msg(tried: Optional[str]) -> str:
47          return (
48              "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
49              f"  Remedy:\n"
50              f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
51              f"  2. Then {_SYNC_REMEDY}\n"
52              "  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive,
macOS "
53              "~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
54              f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
55          )
56
57
58      def _not_synced_msg(root: str, rel: str, uri: str) -> str:
59          return (
60              f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
```

37. assistant (2026-06-21T10:20:16.318Z)

This is decisive for the STORAGE_INCOHERENT question. The `gdrive` backend is explicitly **a

locally-mounted filesystem** (`gdrive://` ? `<mount>/My Drive/...`, "everything else is plain local-filesystem I/O") and the module docstring states a headless box (GPU VM, Cloud Run) has **NO mount** ? so `gdrive` genuinely cannot serve in a cloud-serving deployment. Excluding it from the cloud allowlist is **correct**, not a false-positive. So the `(gcs, s3)` allowlist for the fatal check is right given the 4-backend Literal universe: only `gcs`/`s3` are truly remote-fetchable on a headless box; `local` and `gdrive` are both local-FS. The fatal check is sound and cannot false-positive within the constrained backend universe.

Now I have enough to assess all four lens questions. Let me do a final check on the `CREDS_UNCONFIRMED` false-negative breadth by confirming `GOOGLE_CLOUD_PROJECT` / `CLOUDSDK_CONFIG` semantics, and confirm s3 has no equivalent creds warning (a deliberate asymmetry vs a gap).

38. user (2026-06-21T10:20:17.258Z)

```
1       """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3       One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4       only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5       ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7       Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8       style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9       / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10      (tests pass a fake; production lets the constructor build one via boto3).
11      """
12      from __future__ import annotations
13
14      import os
15      from typing import Any, Iterator, Optional
16
17      from backend.storage.base import StorageBackend
18      from backend.storage.ref import StorageRef, StorageStat
19
20
21      class S3Storage(StorageBackend):
22          def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                       region: Optional[str] = None, addressing_style: str = "auto",
24                       signed_url_ttl: int = 3600, **_ignored) -> None:
25              self._ttl = signed_url_ttl
26              if client is not None:
27                  self._client = client
28                  return
29              import boto3  # type: ignore  # lazy: only when an s3:// URI is actually used
30              from botocore.config import Config  # type: ignore
31              self._client = boto3.client(
32                  "s3",
33                  endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
34                  region_name=region,
35                  config=Config(s3={"addressing_style": addressing_style}),
36              )
37
38          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
39              if dst is None:
40                  raise ValueError("S3Storage.fetch_to_local requires a dst path")
41              dst = str(dst)
42              if os.path.exists(dst) and not overwrite:
43                  return dst
44              os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
45              self._client.download_file(ref.bucket, ref.key, dst)
```

39. assistant (2026-06-21T10:20:27.323Z)

I now have full grounding on all four lens dimensions. Let me synthesize the findings.

Summary of grounded analysis:

1. **STORAGE_INCOHERENT allowlist `(gcs, s3)`** ? backend universe is the Pydantic `Literal["local","s3","gcs","gdrive"]`. `gdrive` is explicitly a locally-mounted FS (cannot serve on a headless cloud box, per its own docstring). So the fatal check is correct and cannot false-positive. **No issue.**

2. **CUDA warnings probe wrong process** ? real concern. The worker-process `probe_cuda_available()` probes torch in the main worker process, but actual detection (and its device resolution) happens in a WSL/native **subprocess with its own torch env**. `device.py` itself documents `cuda_available` is "probed in the runner's own torch env (not the main process)". So `CPU_FALLBACK`/`CUDA_NOT_VISIBLE` can fire on a GPU box. The WARNING-not-error choice is honest, but the `CPU_FALLBACK` message wording over-claims.

3. **CREDS_UNCONFIRMED hints** ? minor false-negative breadth (`GOOGLE_CLOUD_PROJECT`, `CLOUDSDK_CONFIG` commonly set without resolvable ADC); plus s3 has no creds heads-up at all (asymmetry). Both WARNING-only, so low severity.

4. **Worker torch-import cost on default-OFF** ? real, measured. ~3s eager torch import on every `cmd_infer`/`cmd_train` even when `profile==""`, because the probe is evaluated as a call argument before the profile gate. Pre-M5 worker never imported torch.

40. user (2026-06-21T10:20:53.204Z)

Structured output provided successfully

41. assistant (2026-06-21T10:21:07.089Z)

I've completed the false-positive/false-negative review of the M5 startup checks. Summary of grounded findings:

Major (worker default-OFF cost regression): `backend/worker/__main__.py:102` ? `probe_cuda_available()` is passed as a positional argument to `enforce_startup_checks(...)`, so Python evaluates it *before* the function runs, *before* the profile gate inside `run_startup_checks`. It therefore runs on every `cmd_infer`/`cmd_train` including `profile==""`. Measured locally at ~3.1s (lazy torch import + `torch.cuda.is_available()`), and the pre-M5 worker never imported torch at all (detection is delegated to a WSL/native subprocess). So default-OFF is output-identical but gains a ~3s torch+CUDA-init cost per invocation. The server path is fine (passes `cuda_available=None`).

Minor (CUDA warning probes the wrong process): The worker probes torch in the *main* process, but actual device resolution runs in the detector's separate subprocess env (WSL conda `wasb`, or native `WASB_PYTHON`) ? `device.py:36` itself documents `cuda_available` is "probed in the runner's own torch env (not the main process)". So `CPU_FALLBACK`/`CUDA_NOT_VISIBLE` can fire on a real GPU box. WARNING-not-error is the right call and most messages hedge correctly ("visible to this process"), but the `CPU_FALLBACK` message over-claims by asserting as fact "resolved to CPU ... Inference will run on CPU".

Nit (creds hints): `GOOGLE_CLOUD_PROJECT` / `CLOUDSDK_CONFIG` in `_GCP_ENV_HINTS` are commonly set without resolvable ADC, so they can suppress `CREDS_UNCONFIRMED` falsely; and s3 has no parallel creds heads-up (asymmetry). Both warning-only, low impact.

Not an issue ? STORAGE_INCOHERENT (the one fatal): The backend universe is the Pydantic `Literal["local","s3","gcs","gdrive"]`. `gdrive` is explicitly a locally-mounted filesystem that cannot serve on a headless cloud box (per `backend/storage/gdrive.py:1-21`), so the `(gcs, s3)` cloud allowlist is correct ? a legitimate cloud-serving config cannot trip it wrongly.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\startup.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\presets.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\device.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py`.